

Pocket-Pixel

Documentation for Pixel Pocket

Sakshat Sachdeva, Parth Goyal, Nischay Khade, Asher Ul Haque

March 23, 2025

Team Details

- Sakshat Sachdeva - 2023474
- Parth Goyal - 2023371
- Nischay Khade - 2023352
- Asher Ul Haque - 2023151

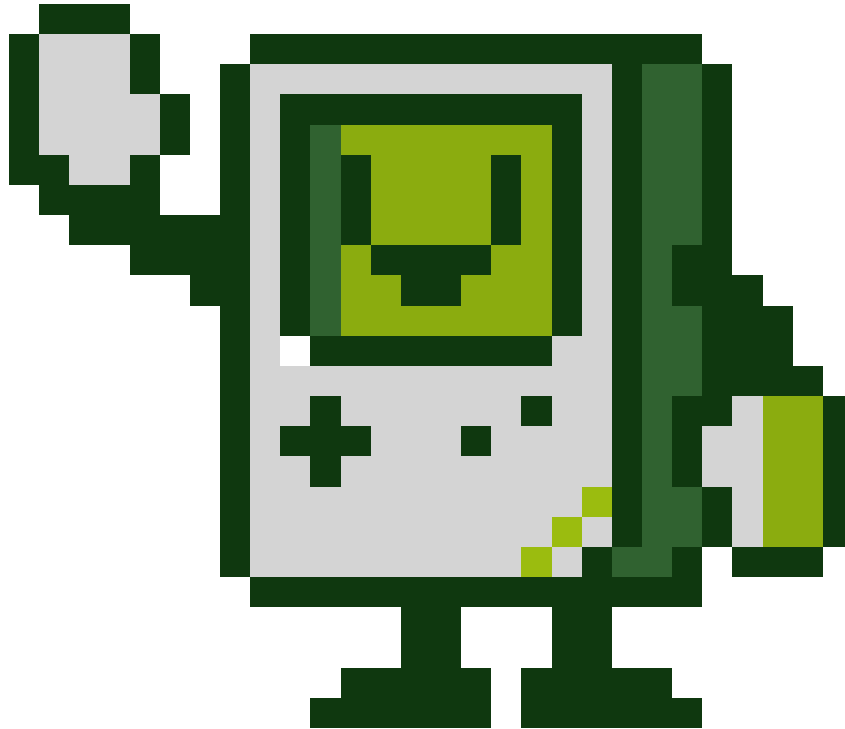


Figure 1: Team Members

0.1 Business Requirements

Pocket-Pixel is a GameBoy emulator designed for both Android and web platforms, offering users a retro gaming experience on modern devices. The system enables users to load, play, and manage GameBoy ROMs efficiently while ensuring high compatibility and performance.

0.2 Features

0.2.1 Cross-Platform Gameplay

- **Android App:** Play GameBoy games on the go with optimized touch controls.
- **Web App:** Enjoy in-browser gaming using WebAssembly for smooth performance.
- **High-Performance Emulator Core:** Built using C/C++ for efficient rendering and accurate emulation.

0.2.2 ROM Management

- **Server Integration:** Download popular ROMs directly within the Android app for offline play.
- **Find Any ROM:** A vast collection ensures availability of most ROMs.

0.2.3 Cloud Saves

- **Save Progress Anywhere:** Sync game saves to a secure web server using PostgreSQL.
- **Cross-Device Continuity:** Resume gameplay seamlessly between different devices.

0.2.4 Competitive Gaming

- **Leaderboards:** Compete globally on metrics like high scores and hours played.
- **Community Challenges:** Participate in unique challenges with other players.

0.2.5 Game Discovery

- **Smart Recommendations:** Machine learning-based suggestions for new games.
- **Trending Titles:** Stay updated with popular retro games.

0.3 Entity Relationship Diagram

The following image represents the ER diagram for the Pocket-Pixel system:

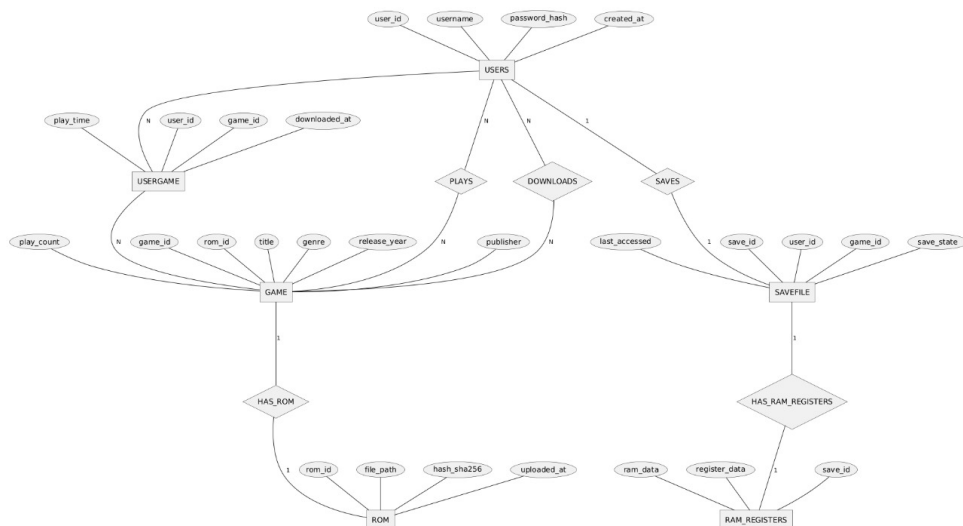


Figure 2: ER Diagram of the Pocket-Pixel System

0.4 SQL queries

The files submitted consisted of three files. First is to create the SQL table. Second is to insert into SQL table. Third is the queries that we will be using to fetch data from the database.

0.4.1 Creating the table

```
1 -- This file creates the database, tables etc in Pixel Pocket backend
2
3 --WARNING:: CREATE THE DATABASE AND CONNECT BEFORE STARTING
4 CREATE DATABASE pixel_pocket; -- Create the database
5 \c pixel_pocket
6
7 -- NOTE: CREATE ALL THE TABLES
8
9 -- The Gamers
10 -- The rest of the group wants to call this Users, BUT CREATE TABLE GAMER
11 -- GAMERS sounds (slay emoji) (I blame Asher for this). I am mentioning the names that the rest
12 -- of the group will use anyways-- GAMERS sounds (slay emoji).
13 CREATE TABLE GAMER
```

```

14 (
15     -- Attributes
16     gamer_id SERIAL, -- user_id
17     gamer_tag VARCHAR(30) UNIQUE NOT NULL, -- username
18     password_hash TEXT NOT NULL, -- stores the password but not exactly
19     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- Cant use Now() for some reason
20     play_time INTERVAL DEFAULT '0', -- How long has the user used the app.
21     play_count INT DEFAULT 0, -- How many games has the gamer played
22
23     -- Keys
24     PRIMARY KEY (gamer_id)
25 );
26
27 -- The Games I want to call this Cartridges, but I have been far less successful in convincing
28 -- the grp of this
29 CREATE TABLE GAME
30 (
31     -- Attributes
32     game_id SERIAL, -- game_id
33     title VARCHAR(255) NOT NULL, -- title of the game eg: Pokemon Red
34     release_year INT DEFAULT 1989
35         CHECK(release_year >= 1989), -- Year released (not before 1989)
36     publisher VARCHAR(100), -- Who made the game? Capcop? Nintendo?
37     box_art BYTEA, -- A cover art for the game
38
39     -- Keys
40     PRIMARY KEY (game_id)
41 );
42
43 -- Game from Gamer POV
44 CREATE TABLE GAMER_POV -- The rest of the group wants to call this USERGAME, eww
45 (
46     -- Attributes
47     gamer_id INT,
48     game_id INT,
49     downloaded_at TIMESTAMP DEFAULT NULL, -- The user could simply play without downloading
50     play_time INTERVAL DEFAULT '0', -- How long has this game been played
51     play_count INT DEFAULT 0, -- How many times has this game been played
52
53     -- Keys
54     PRIMARY KEY (gamer_id, game_id),
55     FOREIGN KEY (gamer_id)
56         REFERENCES GAMER(gamer_id)
57         ON DELETE CASCADE,
58     FOREIGN KEY (game_id)
59         REFERENCES GAME(game_id)
60         ON DELETE CASCADE
61 );
62
63 -- SAVES TABLE (Stores user save states)
64 CREATE TABLE SAVES
65 (
66     -- Attributes
67     save_id SERIAL,
68     gamer_id INT,
69     game_id INT,
70     slot_number INT DEFAULT 1, -- Allows multiple save slots
71     save_state BYTEA NOT NULL, -- The actual save file (or file path)
72     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- In case we have multiple saves
73
74     -- Keys
75     PRIMARY KEY (save_id),
76     FOREIGN KEY (gamer_id)
77         REFERENCES GAMER(gamer_id)
78         ON DELETE CASCADE,
79     FOREIGN KEY (game_id)
80         REFERENCES GAME(game_id)
81         ON DELETE CASCADE
82 );
83
84 -- Components (This is like an abstract class)
85 CREATE TABLE COMPONENT
86 (

```

```

87  -- Attributes
88  component_id SERIAL,
89  type CHAR(3) NOT NULL CHECK (type IN ('ROM', 'RAM')), -- What kind of component
90
91  -- Keys
92  PRIMARY KEY (component_id)
93 );
94
95 -- ROM (The assembly instructions of the game) - subtype of COMPONENT
96 CREATE TABLE ROM
97 (
98  -- Attributes
99  component_id INT,
100  file_path TEXT NOT NULL, -- Where is the file located
101  hash_sha256 TEXT UNIQUE NOT NULL, -- Ensure the game is genuine
102  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- When was it uploaded
103
104  -- Keys
105  PRIMARY KEY (component_id),
106  FOREIGN KEY (component_id)
107    REFERENCES COMPONENT(component_id)
108    ON DELETE CASCADE
109 );
110
111 -- RAM registers (state of the game)
112 CREATE TABLE RAM_REGISTER
113 (
114  -- Attributes
115  component_id INT,
116  register_file BYTEA NOT NULL, -- Might change to one attribute per register in the future
117  save_id INT DEFAULT NULL, -- A game might not be saved yet
118
119  -- Keys
120  PRIMARY KEY (component_id),
121  FOREIGN KEY (component_id)
122    REFERENCES COMPONENT(component_id)
123    ON DELETE CASCADE,
124  FOREIGN KEY (save_id)
125    REFERENCES SAVES(save_id)
126    ON DELETE CASCADE
127 );
128
129 -- Game Components (links a game to its components)
130 CREATE TABLE GAME_COMPONENTS
131 (
132  -- Attributes
133  game_id INT, -- Which game
134  component_id INT, -- What component
135
136  -- Keys
137  PRIMARY KEY (game_id, component_id), -- Unique match required
138  FOREIGN KEY (game_id)
139    REFERENCES GAME(game_id)
140    ON DELETE CASCADE,
141  FOREIGN KEY (component_id)
142    REFERENCES COMPONENT(component_id)
143    ON DELETE CASCADE
144 );

```

0.4.2 Inserting in the table

```

1  -- Sample Insert Script for Pocket-Pixel Database
2  -- These are sample entries. Others may install a similar system.
3
4  -- Insert sample data into GAMER table (Users)
5  INSERT INTO GAMER (gamer_tag, password_hash, created_at, play_time, play_count)
6  VALUES
7  ('Sakshat123', 'hashed_password_1', '2025-02-15 10:00:00', '1 hour'::interval, 5),
8  ('ParthG', 'hashed_password_2', '2025-02-10 09:30:00', '3 hours'::interval, 12),
9  ('NischayK', 'hashed_password_3', '2025-02-12 11:15:00', '5 hours'::interval, 20),
10 ('AsherH', 'hashed_password_4', '2025-02-11 14:00:00', '2 hours'::interval, 8);

```

```

11
12 -- Insert sample data into GAME table
13 INSERT INTO GAME (title, release_year, publisher, box_art)
14 VALUES
15     ('Pokemon Red', 1996, 'Nintendo', NULL),
16     ('The Legend of Zelda', 1993, 'Nintendo', NULL),
17     ('Tetris', 1989, 'Bullet-Proof Software', NULL),
18     ('Super Mario Land', 1989, 'Nintendo', NULL);
19
20 -- Insert sample data into GAMER_POV table (linking GAMERs and GAMES)
21 INSERT INTO GAMER_POV (gamer_id, game_id, downloaded_at, play_time, play_count)
22 VALUES
23     (1, 1, '2025-02-10 15:30:00', '30 minutes'::interval, 2),
24     (1, 2, '2025-02-12 16:00:00', '1 hour'::interval, 3),
25     (2, 3, '2025-02-13 14:00:00', '2 hours'::interval, 5),
26     (3, 4, NULL, '45 minutes'::interval, 1);
27
28 -- Insert sample data into SAVES table (User Save States)
29 INSERT INTO SAVES (gamer_id, game_id, slot_number, save_state, created_at)
30 VALUES
31     (1, 1, 1, '\x3fa5b8e3c7d91f2a', '2025-02-14 12:00:00'),
32     (1, 2, 1, '\x3fa5b8e3c7d91f4a', '2025-02-14 13:00:00'),
33     (2, 3, 1, '\x3fa5b8e3c7d91f3a', '2025-02-14 14:00:00');
34
35 -- Insert sample data into COMPONENT table
36 INSERT INTO COMPONENT (type)
37 VALUES
38     ('ROM'),
39     ('ROM'),
40     ('RAM');
41
42 -- Insert sample data into ROM table (Subtype of COMPONENT)
43 INSERT INTO ROM (component_id, file_path, hash_sha256, uploaded_at)
44 VALUES
45     (1, '/roms/pokemon_red.gbc', 'hash_value_1', '2025-02-10 10:00:00'),
46     (2, '/roms/zelda.gbc', 'hash_value_2', '2025-02-11 12:00:00');
47
48 -- Insert sample data into RAM_REGISTER table (Subtype of COMPONENT)
49 INSERT INTO RAM_REGISTER (component_id, register_file, save_id)
50 VALUES
51     (3, '\x3fa5b8e3c7d91f2a', 1);
52
53 -- Insert sample data into GAME_COMPONENTS table (linking games with components)
54 INSERT INTO GAME_COMPONENTS (game_id, component_id)
55 VALUES
56     (1, 1), -- 'Pokemon Red' uses ROM component (component_id 1)
57     (2, 2), -- 'The Legend of Zelda' uses ROM component (component_id 2)
58     (3, 3); -- 'Tetris' uses RAM component (component_id 3)

```

0.4.3 Queries

- Query 1: Find all gamers and their total play time across all games. (We use Join, Aggregate, Group By)

```

pixel_pocket=# SELECT
    g.gamer_tag,
    SUM(gp.play_time) AS total_play_time
FROM
    GAMER g
JOIN
    GAMER_POV gp ON g.gamer_id = gp.gamer_id
GROUP BY
    g.gamer_tag
ORDER BY
    total_play_time DESC;
gamer_tag | total_play_time
-----+-----
ParthG    | 02:00:00
Sakshat123 | 01:30:00
NischayK  | 00:45:00
(3 rows)

```

- Query 2: List all games published by 'Nintendo' released after 1990. (We use Where clause, Select from GAME)

```

pixel_pocket=# SELECT
    title,
    release_year
FROM
    GAME
WHERE
    publisher = 'Nintendo' AND release_year > 1990;
title | release_year
-----+-----
Pokemon Red | 1996
The Legend of Zelda | 1993
(2 rows)

```

- Query 3: Show gamer tags and the titles of the games they have downloaded, ordered by gamer tag and download date. (Join, Order By)

```

pixel_pocket=# SELECT
    g.gamer_tag,
    game.title,
    gp.downloaded_at
FROM
    GAMER g
JOIN
    GAMER_POV gp ON g.gamer_id = gp.gamer_id
JOIN
    GAME ON gp.game_id = game.game_id
WHERE gp.downloaded_at IS NOT NULL
ORDER BY
    g.gamer_tag,
    gp.downloaded_at;

```

gamer_tag	title	downloaded_at
ParthG	Tetris	2025-02-13 14:00:00
Sakshat123	Pokemon Red	2025-02-10 15:30:00
Sakshat123	The Legend of Zelda	2025-02-12 16:00:00

(3 rows)

- Query 4: Find games that have not been played by any gamer. (Subquery - using NOT IN)

```

pixel_pocket=# SELECT
    title
FROM
    GAME
WHERE
    game_id NOT IN (SELECT game_id FROM GAMER_POV);

```

title

(0 rows)

- Query 5: Calculate the average play time for each game across all gamers who have played it. (Aggregate, Group By, Join)


```

pixel_pocket=# SELECT
    game.title,
    AVG(gp.play_time) AS average_play_time
FROM
    GAME
JOIN
    GAMER_POV gp ON game.game_id = gp.game_id
GROUP BY
    game.title
ORDER BY
    average_play_time DESC;

```

title	average_play_time
Tetris	02:00:00
The Legend of Zelda	01:00:00
Super Mario Land	00:45:00
Pokemon Red	00:30:00

(4 rows)

- Query 6: List gamers who have played more than 5 games. (Aggregate, Group By, Having, Join)

```

pixel_pocket=# SELECT
    g.gamer_tag,
    COUNT(DISTINCT gp.game_id) AS number_of_games_played
FROM
    GAMER g
JOIN
    GAMER_POV gp ON g.gamer_id = gp.gamer_id
GROUP BY
    g.gamer_tag
HAVING
    COUNT(DISTINCT gp.game_id) > 1;

```

gamer_tag	number_of_games_played
Sakshat123	2

(1 row)

- Query 7: Find the game with the most saves. (Aggregate, Group By, Join, Order By, Limit)

```

pixel_pocket=# SELECT
    game.title,
    COUNT(s.save_id) AS number_of_saves
FROM
    GAME
JOIN
    SAVES s ON game.game_id = s.game_id
GROUP BY
    game.title
ORDER BY
    number_of_saves DESC
LIMIT 1;

```

title	number_of_saves
Pokemon Red	1

(1 row)

- Query 8: Show gamers who have saves for more than one game. (Aggregate, Group By, Having, Join)

```

pixel_pocket=# SELECT
    g.gamer_tag,
    COUNT(DISTINCT s.game_id) AS number_of_games_saved
FROM
    GAMER g
JOIN
    SAVES s ON g.gamer_id = s.gamer_id
GROUP BY
    g.gamer_tag
HAVING
    COUNT(DISTINCT s.game_id) > 0;

```

gamer_tag	number_of_games_saved
Nischayk	1
ParthG	1
Sakshat123	1

(3 rows)

- Query 9: Find the latest save created for each game and gamer combination. (Window Function - ROW_NUMBER())

```

pixel_pocket=# SELECT
    g.gamer_tag,
    game.title,
    s.created_at AS latest_save_time
FROM (
    SELECT
        gamer_id,
        game_id,
        created_at,
        ROW_NUMBER() OVER(PARTITION BY gamer_id, game_id ORDER BY created_at DESC) as rn
    FROM
        SAVES
) s_ranked
JOIN GAMER g ON s_ranked.gamer_id = g.gamer_id
JOIN GAME game ON s_ranked.game_id = game.game_id
JOIN SAVES s ON s_ranked.game_id = s.save_id
WHERE s_ranked.rn = 1
ORDER BY g.gamer_tag, game.title;
gamer_tag | title | latest_save_time
-----+-----+-----
ParthG | Tetris | 2025-02-14 14:00:00
Sakshat123 | Pokemon Red | 2025-02-14 12:00:00
Sakshat123 | The Legend of Zelda | 2025-02-14 13:00:00
(3 rows)

```

- Query 10: List all ROM components and the games they belong to. (Join, Select specific columns)

```

pixel_pocket=# SELECT
    r.file_path,
    game.title
FROM
    ROM r
JOIN
    COMPONENT c ON r.component_id = c.component_id
JOIN
    GAME_COMPONENTS gc ON c.component_id = gc.component_id
JOIN
    GAME game ON gc.game_id = game.game_id;
file_path | title
-----+-----
/roms/pokemon_red.gbc | Pokemon Red
/roms/zelda.gbc | The Legend of Zelda
(2 rows)

```

- Query 11: Find gamers who have played games released before 1995. (Join, Subquery in WHERE clause, EXISTS)

```

pixel_pocket=# SELECT DISTINCT
    g.gamer_tag
FROM
    GAMER g
WHERE EXISTS (
    SELECT 1
    FROM GAMER_POV gp
    JOIN GAME game ON gp.game_id = game.game_id
    WHERE gp.gamer_id = g.gamer_id AND game.release_year < 1995
);
gamer_tag
-----
NischayK
Sakshat123
ParthG
(3 rows)

```

- Query 12: Get the count of games for each publisher. (Aggregate, Group By)

```

pixel_pocket=# SELECT
    publisher,
    COUNT(*) AS game_count
FROM
    GAME
GROUP BY
    publisher
ORDER BY
    game_count DESC;
    publisher | game_count
-----+-----
    Nintendo |          3
Bullet-Proof Software |          1
(2 rows)

```

- Query 13: Find games with no associated ROM components. (LEFT JOIN, WHERE IS NULL)

```

pixel_pocket=# SELECT
    game.title
FROM
    GAME
LEFT JOIN
    GAME_COMPONENTS gc ON game.game_id = gc.game_id
LEFT JOIN
    COMPONENT c ON gc.component_id = c.component_id AND c.type = 'ROM'
WHERE
    c.component_id IS NULL;
    title
-----
    Tetris
    Super Mario Land
(2 rows)

```

- Query 14: Get gamer tag and play count for gamers who have played games more than 10 times in total. (Aggregate, Group By, Having, Join)

```
pixel_pocket=# SELECT
g.gamer_tag,
SUM(gp.play_count) AS total_play_count
FROM
  GAMER g
JOIN
  GAMER_POV gp ON g.gamer_id = gp.gamer_id
GROUP BY
  g.gamer_tag
HAVING
  SUM(gp.play_count) > 4
ORDER BY total_play_count DESC;
gamer_tag | total_play_count
-----+-----
Sakshat123 | 5
ParthG | 5
(2 rows)
```

0.5 How LLMs generated wrong queries

We made LLMs generate queries for our database. For now, we used 2 queries. We will be comparing where the LLM went wrong, and how it went wrong.

0.5.1 Query 1

Correct Query:

```
1 SELECT
2   g.gamer_tag,
3   game.title
4 FROM
5   GAMER g
6 JOIN
7   GAMER_POV gp ON g.gamer_id = gp.gamer_id
8 JOIN
9   GAME game ON gp.game_id = game.game_id
10 WHERE gp.downloaded_at IS NOT NULL
11 ORDER BY
12   g.gamer_tag;
```

LLM Generated Incorrect Query:

```
1 SELECT
2   u.username, -- LLM used 'username' instead of 'gamer_tag'
3   g.game_title -- LLM used 'game_title' instead of 'title'
4 FROM
5   Users u -- Incorrect table name/alias; should be GAMER
6 JOIN
7   UserGame ug ON u.user_id = ug.user_id -- Incorrect table (UserGame) and column (user_id) names
8 JOIN
9   Games g ON ug.game_id = g.game_id -- Incorrect table name; should be GAME
10 WHERE ug.downloaded = TRUE -- Incorrect condition; should check for NOT NULL in 'downloaded_at'
11 ORDER BY
12   u.username;
```

Why the LLM Query is Incorrect:

- **Incorrect Table and Column Names:** The LLM used `Users` and `Games` instead of the correct `GAMER` and `GAME`. It also used `username`, `game_title`, and `user_id` instead of the proper `gamer_tag`, `title`, and `gamer_id`.
- **Incorrect Downloaded Condition:** The LLM assumes a boolean column `downloaded` to indicate if a game was downloaded, while the actual schema uses a `downloaded_at` timestamp field and checks for a `NOT NULL` value.

0.5.2 Query 2

Correct Query:

```
1 SELECT DISTINCT
2   game.title
3 FROM
4   GAME
5 JOIN
6   SAVES s ON game.game_id = s.game_id
7 WHERE
8   s.gamer_id IN (
9     SELECT gamer_id
10    FROM GAMER_POV
11    GROUP BY gamer_id
12    HAVING COUNT(DISTINCT game_id) > 3
13  );
```

LLM Generated Incorrect Query:

```
1 SELECT
2   g.title -- LLM might correctly get game title
3 FROM
4   Games g -- Assumes 'Games' instead of GAME
5 WHERE EXISTS (
6   SELECT 1
7   FROM Gamers gamer -- Assumes 'Gamers' instead of GAMER
8   JOIN Saves save ON gamer.gamer_id = save.gamer_id -- Correct join on Saves, but table names are off
9   GROUP BY gamer.gamer_id -- Groups by gamer correctly
10  HAVING COUNT(DISTINCT save.game_id) > 3 -- Incorrectly counts distinct save.game_id instead of
11    GAMER_POV.game_id
12  AND save.game_id = g.game_id -- Attempts to link save.game_id to game title, but logic is flawed
13 );
```

Why the LLM Query is Incorrect:

- **Incorrect Table Names:** The LLM used `Games` and `Gamers` instead of the correct `GAME` and `GAMER`.
- **Incorrect Counting:** The LLM counts distinct `save.game_id` from the `Saves` table rather than using the data from `GAMER_POV` to determine the number of games played by a gamer.
- **Flawed EXISTS Logic:** The subquery uses `EXISTS` and attempts to link `save.game_id` to `g.game_id`, which is not the intended logic for filtering gamers who have played more than 3 games.

0.6 Sources

The following sources were used in developing the Pocket-Pixel project:

- ChatGPT
- MIT License
- Emulator Games
- YouTube

0.7 Changelogs

The following are the change logs made:

- Added SQL files and screenshot proofs. 23/03
- Added how LLMs just mess things up.